

INTRODUÇÃO A PROJETO ORIENTADO A OBJETO



Prof^a. Cristiane Aparecida Lana

AGENDA

1. Introdução a orientação a objetos
2. História da OO
3. Mudança de Enfoque
4. Vantagens da POO
5. Fases do Modelo de Processo de Software
6. Conceitos Básicos de Orientação Objeto
 - Classe, Objeto, Atributo e Método
7. Princípios Básicos de orientação Objeto
 - Abstração, Encapsulamento, Polimorfismo e Herança



O QUE É ORIENTAÇÃO A OBJETO



INTRODUÇÃO A ORIENTAÇÃO A OBJETOS

O QUE É ORIENTAÇÃO A OBJETOS?

- A orientação a objetos (OO), também conhecida como Programação Orientada a Objetos (POO) ou ainda em inglês *Object-Oriented Programming (OOP)*
- é um **paradigma** de análise, projeto e programação de sistemas de software
- baseado na **composição** e **interação** entre **diversas unidades** de software chamadas de **objetos**.



INTRODUÇÃO A ORIENTAÇÃO A OBJETOS

- Os **objetos** que **compartilham** uma **mesma interface**, ou seja, **respondem** as **mesmas mensagens**, são agrupados em **classes**.
- **Objeto** é algo **DINÂMICO**: é **criado por alguém**, tem uma **vida**, e **morre** ou **é morto** por alguém. Assim, **durante a execução** do sistema, os **objetos** podem:
 - ser **construídos**
 - **executar ações**
 - ser **destruídos**
 - tornar-se **inacessíveis**



INTRODUÇÃO A ORIENTAÇÃO A OBJETOS

- A OO é uma abordagem que ajuda a definir a estrutura de programas baseada nos conceitos do mundo real, sejam eles reais ou abstratos.
- Assim, é possível criar programas como componentes, separando as partes do sistema por responsabilidades
- Permitindo que essas partes se comuniquem, por meio de mensagens.



HISTÓRICO DA 00



A ORIENTAÇÃO A OBJETOS

HISTÓRICO

- Surge no início da década de 60 com a implementação da linguagem Simula (*Simula67*).
- Desenvolvida na Noruega
- A linguagem era usada para simulações
- Com essa linguagem emerge o conceito de similaridade com o mundo real que pode e deve ser aplicado no processo de pensar e desenvolver sistemas de software.



A ORIENTAÇÃO A OBJETOS

HISTÓRICO

- A implementação da OO foi sistematizada na *Simula68*
- Na década de 80, surge a *Smalltalk*
 - proposta pela Xerox
 - Considerada a responsável por popularizar o uso de OO
- Exemplos de Linguagens OO
 - Java, C#, C++, Object Pascal (Delphi), Ruby, Python, Lisp, etc.



MUDANÇA DE ENFOQUE



MUDANÇA DE ENFOQUE

- Enfoque **tradicional de modelagem**
 - programação estruturada;
 - construção de **sistemas de informação** baseado na **compreensão** desse sistema
 - como **um conjunto** de subprogramas
 - executam **processos** sobre os **dados**;
 - **Programa** = estrutura de dados + algoritmo.



MUDANÇAS DE ENFOQUE

PROGRAMAÇÃO ESTRUTURADA EM C

```
#include "geral.h"
```

```
struct Data {  
    int dia;  
    int mes;  
    int ano;  
};
```

dados

Tipo Abstrato de Dados "Data"

```
struct Data dataNova(int d, int m, int a) {  
    struct Data dt;  
    dt.dia = d;  
    dt.mes = m;  
    dt.ano = a;  
    return(dt);  
}
```

subprogramas

```
struct Data dataHoje() {  
    struct Data dt;  
  
    struct date d;  
    getdate(&d);  
  
    dt.dia = d.da_day;  
    dt.mes = d.da_mon;  
    dt.ano = d.da_year;  
    return(dt);  
}
```

```
char *imprimeData(struct Data *dt) {  
    char sdia[2], smes[2], sano[4], sdt[10];  
    itoa(dt->dia, sdia, 10);  
    itoa(dt->mes, smes, 10);  
    itoa(dt->ano, sano, 10);  
    strcpy(sdt, sdia);  
    strcat(sdt, "/");  
    strcat(sdt, smes);  
    strcat(sdt, "/");  
    strcat(sdt, sano);  
    return(sdt);  
}
```

```
int totalDeDias(struct Data *dt) {  
    int i, ttAnos = (dt->ano - 1) * 365;  
    for (i=1; i<=(dt->ano-1); i++) { // Número de anos bissextos até a data.  
        if (anoBissexto(i) == 1)  
            ttAnos++;  
    }  
    int ttMeses = 0;  
    for (i=1; i<=(dt->mes-1); i++) {  
        ttMeses += diasNoMes(i, dt->ano);  
    }  
    return(ttAnos + ttMeses + dt->dia);  
}
```

```
char *diaDaSemana(struct Data *dt) {  
    char *nomeDiaDaSemana[7] = {"Domingo", "Segunda-Feira", "Terca-Feira",  
        "Quarta-Feira", "Quinta-Feira", "Sexta-Feira", "Sabado"};  
    return(nomeDiaDaSemana[totalDeDias(dt) % 7]);  
}
```

MUDANÇAS DE ENFOQUE

- Enfoque de **modelagem por objetos**
 - programação orientada a objetos
 - observar o mundo como uma coletânea de objetos que interagem entre si
 - apresentam características próprias
 - representadas pelos seus atributos (dados) e operações (processos, ou métodos);
 - **classe** = atributos + métodos (ou operações)



INTRODUÇÃO A ORIENTAÇÃO A OBJETOS

PROGRAMAÇÃO ORIENTADA A OBJETOS

```
import meuPacote.Geral;
```

```
public class Data {  
    private int dia;  
    private int mes;  
    private int ano;
```

atributos

Classe "Data"

```
    public Data(int d, int m, int a) {  
        this.dia = d;  
        this.mes = m;  
        this.ano = a;  
    }
```

métodos

```
    public Data() {  
        java.util.Date hoje = new java.util.Date();  
        this.dia = hoje.getDate();  
        this.mes = hoje.getMonth() + 1;  
        this.ano = hoje.getYear() + 1900;  
    }
```

```
    public int getDia() {  
        return dia;  
    }
```

```
    public void setDia(int dia) {  
        this.dia = dia;  
    }
```

```
    public int getMes() {  
        return mes;  
    }
```

```
    public void setMes(int mes) {  
        this.mes = mes;  
    }
```

```
    public int getAno() {  
        return ano;  
    }
```

```
    public void setAno(int ano) {  
        this.ano = ano;  
    }
```

```
    public String imprimeData() {  
        return(dia + "/" + mes + "/" + ano);  
    }
```

Classe "NovaData"

```
class NovaData extends Data {  
    public NovaData(int d, int m, int a) {  
        super(d, m, a);  
    }  
  
    public NovaData() {  
        super();  
    }  
  
    private int totalDeDias() {  
        int ttAnos = (getAno()-1) * 365;  
        for (int i=1; i<=(getAno()-1); i++) {  
            if (Geral.anoBissexto(i) == true)  
                ttAnos++;  
        }  
  
        int ttMeses = 0;  
        for (int i=1; i<=(getMes()-1); i++) {  
            ttMeses += Geral.diasNoMes(i, getAno());  
        }  
  
        return(ttAnos + ttMeses + getDia());  
    }  
  
    public String diaDaSemana() {  
        String nomeDiaDaSemana[] = {"Domingo", "Segunda-Feira", "Terça-Feira",  
            "Quarta-Feira", "Quinta-Feira", "Sexta-Feira", "Sábado"};  
        return (nomeDiaDaSemana[totalDeDias() % 7]);  
    }  
}
```



MUDANÇA DE ENFOQUE

- **programação estrutura é baseado nas estruturas básicas de controle** (sequencial, condicional e repetição), modularização, tipos abstratos de dados



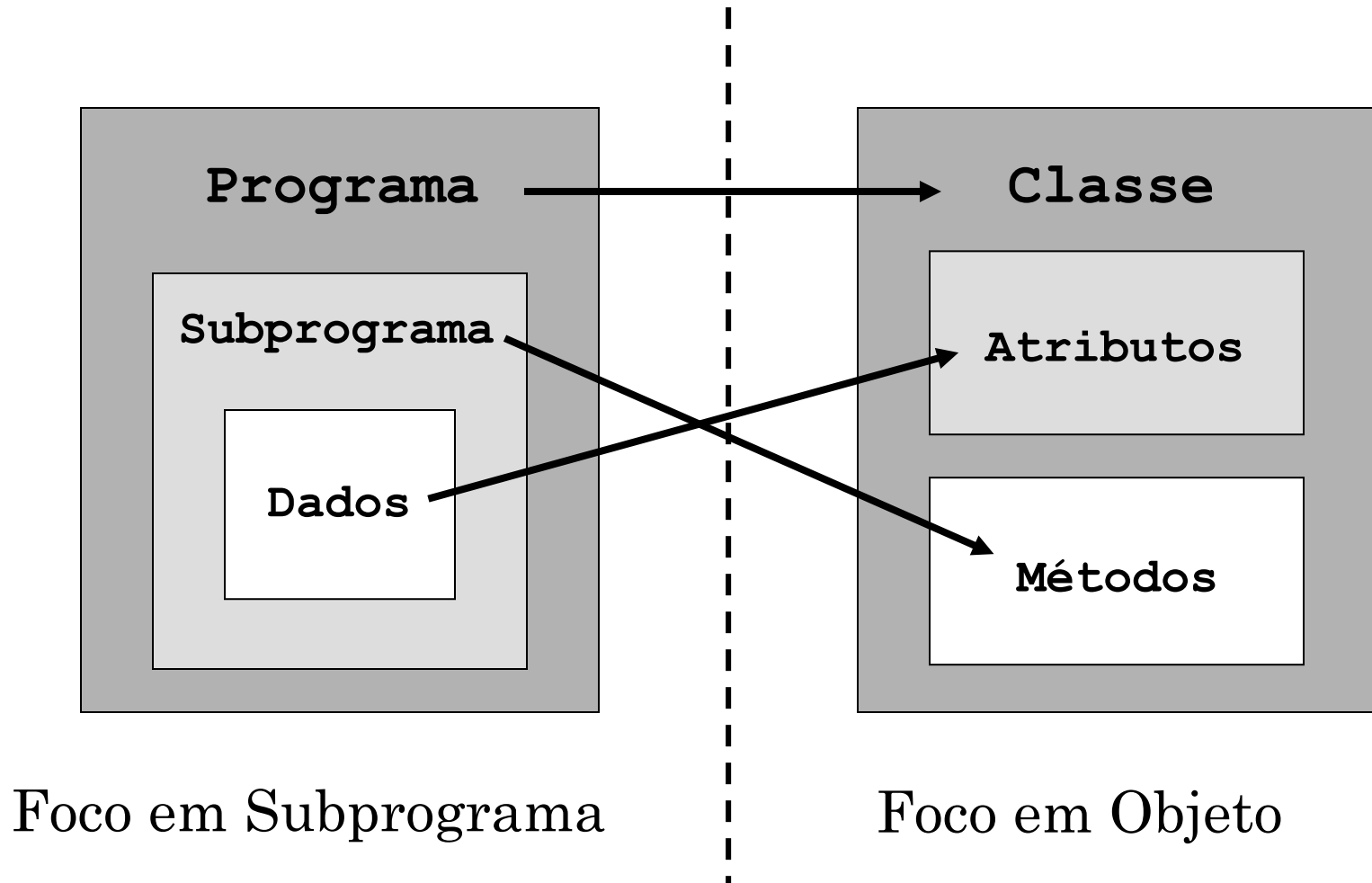
- **programação orientada a objetos:** classes, objeto, herança (reutilização de código), polimorfismo



Enfoque baseado em subprograma

versus

enfoque baseado em objeto



VANTAGENS DA PROGRAMAÇÃO ORIENTADA A OBJETOS



VANTAGENS DA PROGRAMAÇÃO ORIENTADA A OBJETOS

- Abstração de dados:

- os detalhes referentes às representações das classes serão visíveis apenas a seus atributos;

- Reutilização:

- o encapsulamento dos métodos e a representação dos dados usados na construção de classes facilitam o desenvolvimento de software reutilizável, auxiliando na produtividade de sistemas;

- Diminuição da Complexidade:

- as classes delimitam-se em unidades naturais para a alocação de tarefas de desenvolvimento de software;



VANTAGENS DA PROGRAMAÇÃO ORIENTADA A OBJETOS

- Extensibilidade:

- facilidade de estender o software devido a duas razões:
 - herança: novas classes são construídas a partir das que já existem;
 - as classes formam uma estrutura fracamente acoplada o que facilita alterações;

- Mesma notação:

- é utilizada desde a fase de análise até a implementação.

- Manutenibilidade:

- a modularização natural em classes facilita a realização de alterações no software.



FASES DO MODELO DE PROCESSO DE SOFTWARE



FASES DO MODELO

Análise de sistema
Planejamento
Análise de Requisitos

Projeto
Codificação
Teste

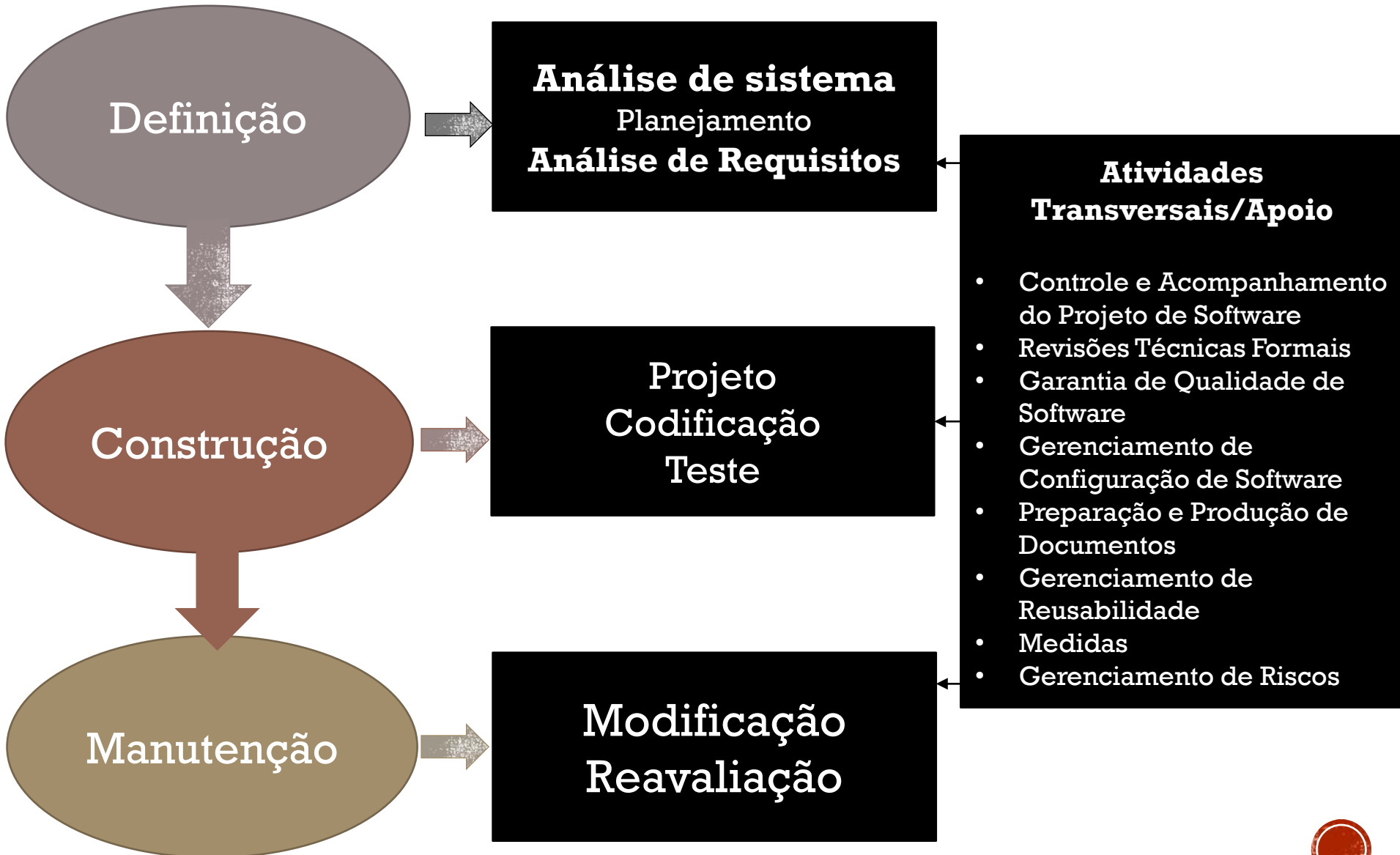
Modificação
Reavaliação



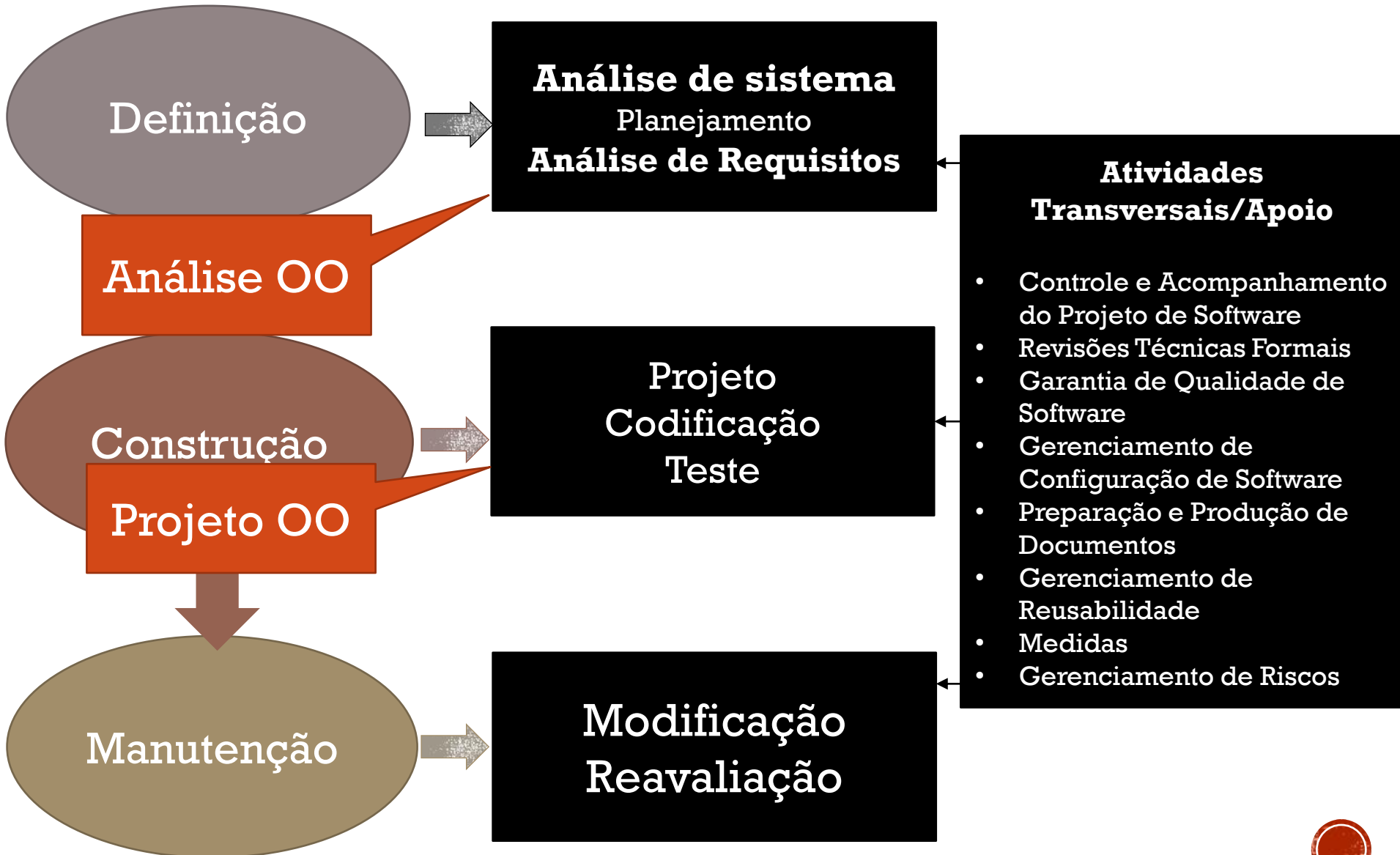
FASES DO MODELO



FASES DO MODELO



FASES DO MODELO

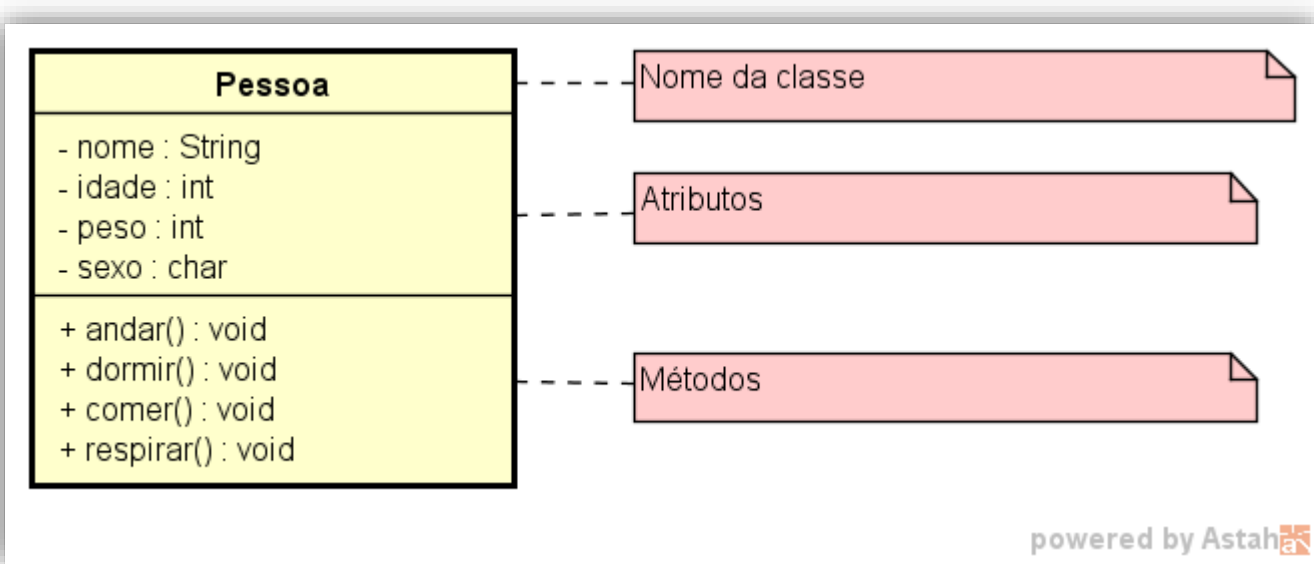


CONCEITOS BÁSICOS DE OO



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

- Classe
- Objetos (Instância)
- Atributos
- Métodos



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

CLASSE

- A **classe** é a implementação de tipo abstrato de dados (TAD) no **paradigma** orientado a objetos.
- A **classe** define **as propriedades** (atributos) e os **comportamentos** (métodos).
- Além disso, **uma classe** define **como produzir** (instanciar) **objetos** a partir dela.



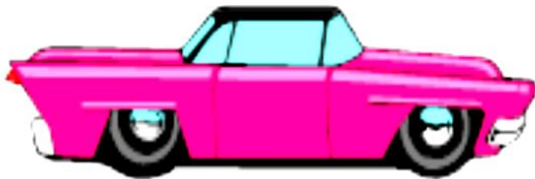
CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

CLASSE

Objeto 1



Objeto 2

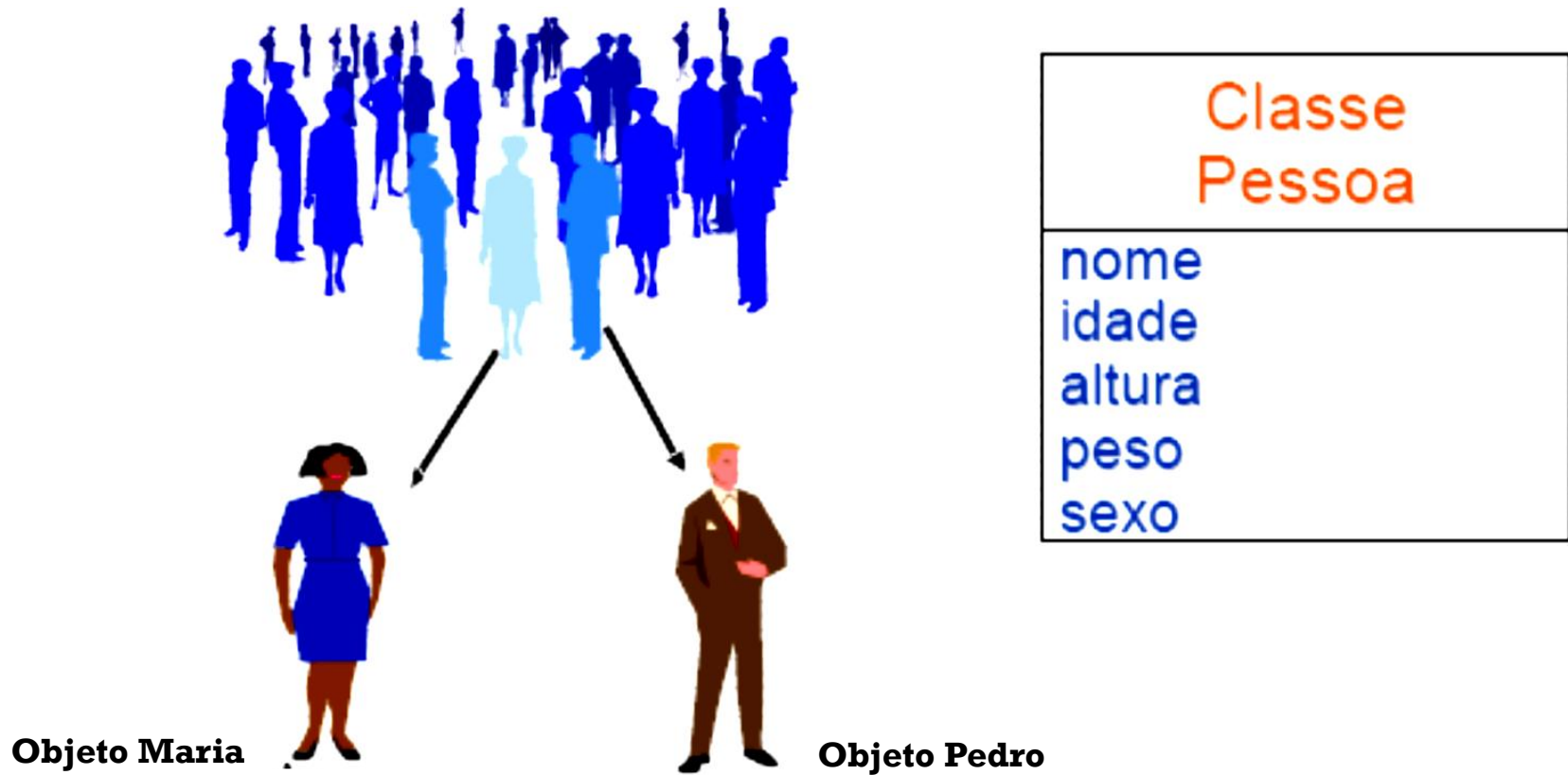


Classe Automovel
numeroPortas cor fabricante ano placa



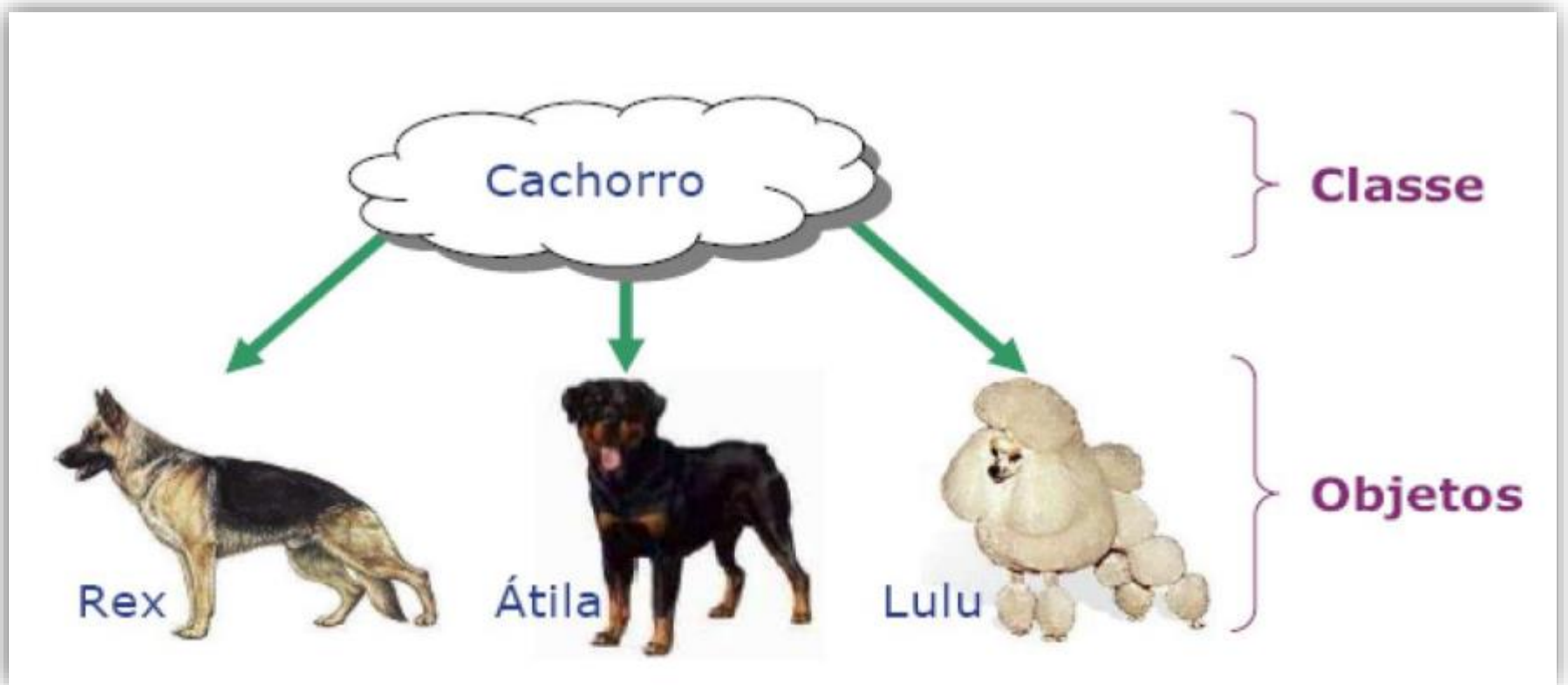
CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

CLASSE



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

CLASSE



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

OBJETO

- Um objeto **é uma construção de software que encapsula estado e comportamento** de propriedades (atributos) e operações (métodos), respectivamente;
 - **Estado de um Objeto:**
 - composto por **suas propriedades** e seus **respectivos valores**;
 - **Comportamento:**
 - a **maneira** como **o objeto reage** quando o **seu estado é alterado** ou quando **uma mensagem é recebida**.



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

OBJETO

```
try {
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery();
    while (rs.next()) {
        Juiz j = new Juiz();
        j.setId(rs.getString("idjuiz"));
        j.setNome(rs.getString("nome"));
        j.setLotacao(rs.getString("lotacao"));
        j.setGrupo(rs.getString("grupo"));
        juizes.add(j);
    }
} catch (SQLException u) {
    new FrameNewDeslocamento().FrameError();
    throw new RuntimeException(u);
}
return juizes;
```

← INSTANCIANDO O
OBJETO J DA CLASSE
JUIZ



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

ATRIBUTO

São **características do objeto**.



Automóvel

Atributos ...

- nº de portas = 2
- cor = vermelho
- ano = 1995
- etc.



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

MÉTODO

- São **subprogramas** com **finalidades específicas**, ou seja, são **as operações** ou **ações realizadas** dentro das classes.
- Para o **nome do método**, é **padrão começar** com letra **minúscula**.
- **Exemplos:**
 - `incluirPessoa();`
 - `pesquisarPeloNome(nome:String);`



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

QUAL A DIFERENÇA ENTRE CLASSE E OBJETO?



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

QUAL A DIFERENÇA ENTRE CLASSE E OBJETO?

- **Classe:**

- é definição **do tipo**;
- representa **um conjunto de objetos** de mesmo tipo;
- Classe = {obj1, obj2, obj3, ..., objN}

contém



esta contido

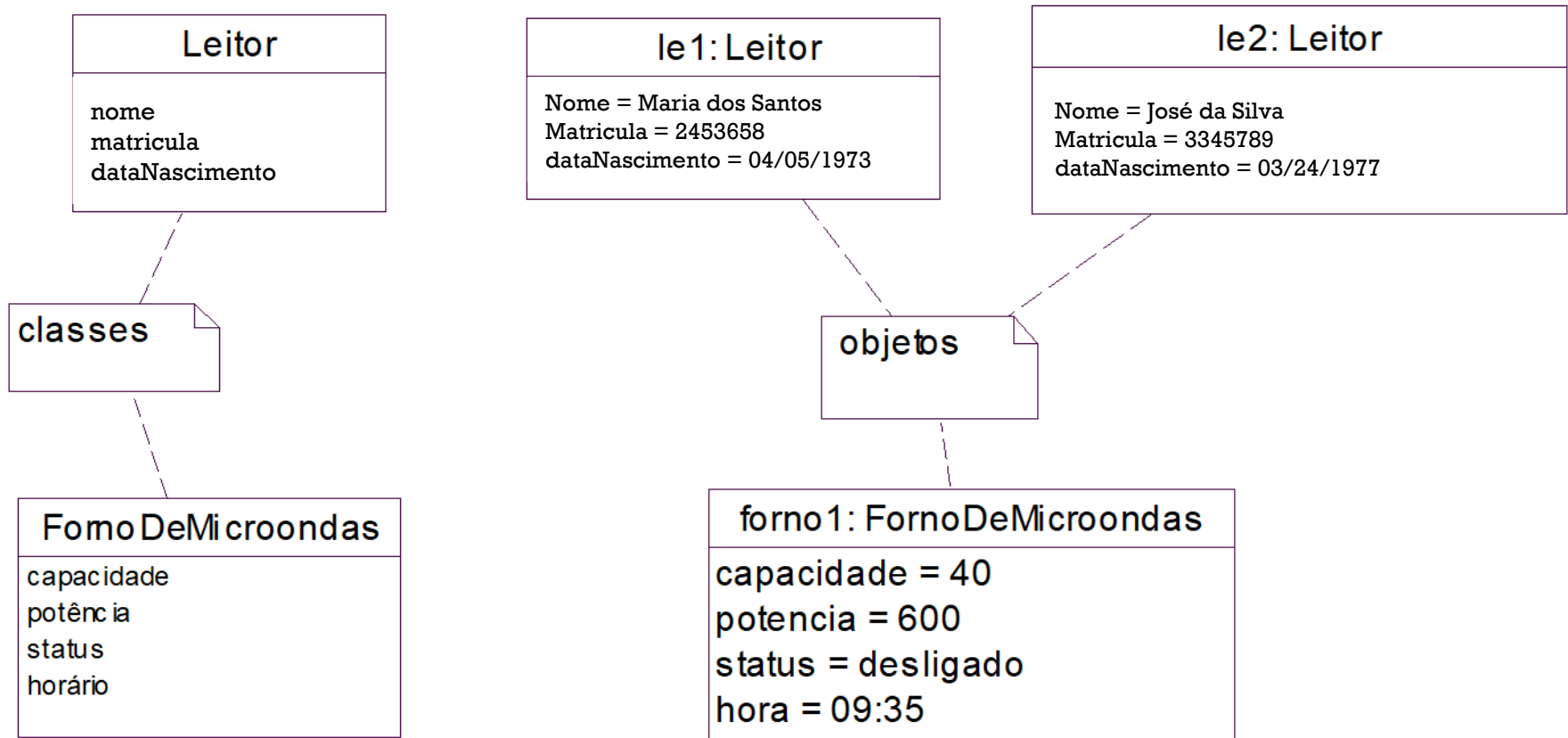
- **Objeto:**

- é **cada instância** derivada da classe;
- é **um elemento** do **conjunto** representado pela **classe**



CONCEITOS BÁSICOS DE ORIENTAÇÃO OBJETO

QUAL A DIFERENÇA ENTRE CLASSE E OBJETO?



PRINCÍPIOS DE ORIENTAÇÃO OBJETO



AGENDA

1. Introdução a orientação a objetos
2. História da OO
3. Mudança de Enfoque
4. Vantagens da POO
5. Fases do Modelo de Processo de Software
6. Conceitos Básicos de Orientação Objeto
 - Classe, Objeto, Atributo e Método
7. Princípios Básicos de orientação Objeto
 - Abstração, Encapsulamento, Polimorfismo e Herança



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Orientação Objeto

Abstração

Encapsulamento

Polimorfismo

Herança



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Abstração

“**Extrair** tudo o que **for essencial** e **nada mais**”

(Aaron Walsh)

“A **abstração** é o processo de **filtragem** de detalhes **sem** **importância** do objeto, para **que apenas** as características **apropriadas** que o **descrevem** permaneçam”

(Peter Van Der Linden)

Conceito aplicado a **criação** de **software** baseado em **objetos**, partindo do **princípio** que **devemos considerar** a **essência** de **cada objeto** e **não** pensar em todos os **detalhes** de implementação;

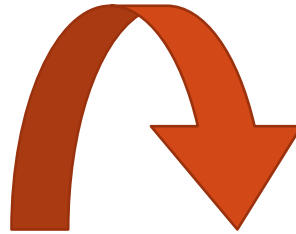


PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Abstração

```
public class Juiz {
```

```
    private String id;  
    private String nome;  
    private String grupo;  
    private String lotacao;  
    private String telefone;  
    private String estadocivil;  
    private String idade;  
    private String endereço;  
    private String dependentes;  
    private String corpreferida;
```



```
public class Juiz {
```

```
    private String id;  
    private String nome;  
    private String grupo;  
    private String lotacao;
```

COMPLEXO E DESNECESSÁRIO

ENXUTO E SUFICIENTE

Vantagem: Robustez



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Abstração

Analise o seguinte programa em linguagem c. e explique o que esse código faz:

```
#include <stdio.h>
#include <stdlib.h>

void main()
{
    int m[3][3], i, j;

    for (i=0; i<3; i++) {
        for (j=0; j<3; j++) {
            scanf("%d", &m[i][j]);
        }
    }

    for (i=0; i<3; i++) {
        for (j=0; j<3; j++) {
            if ((m[i][j] % 2) != 0)
                printf("%d \n", m[i][j]);
        }
    }
}
```



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Abstração

Analise o seguinte programa em linguagem c. e explique o que esse código faz:

```
#include <stdio.h>
#include <stdlib.h>

void main()
{
    int m[3][3], i, j;

    for (i=0; i<3; i++) {
        for (j=0; j<3; j++) {
            scanf("%d", &m[i][j]);
        }
    }

    for (i=0; i<3; i++) {
        for (j=0; j<3; j++) {
            if ((m[i][j] % 2) != 0)
                printf("%d \n", m[i][j]);
        }
    }
}
```

- O código acima traz uma matriz do tipo 3x3, ou seja, três linhas e três colunas. Este código realiza as condições de teste para a linha e para a coluna dentro do for. Posteriormente ele faz a leitura da matriz com o comando "scanf" com os valores de linha e coluna inseridos pelo usuário. - Por fim, este código realiza um teste de condição, onde se o resto da divisão da matriz por 2 for diferente de zero ela será uma matriz ímpar. Se a condição anterior for verdadeira o programa deve imprimir como dado de saída uma matriz do tipo 3x3, cujo todos os seus elementos será de valores ímpares.

Esse código lê a matriz 3 x 3 e imprime somente os números ímpares.

PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

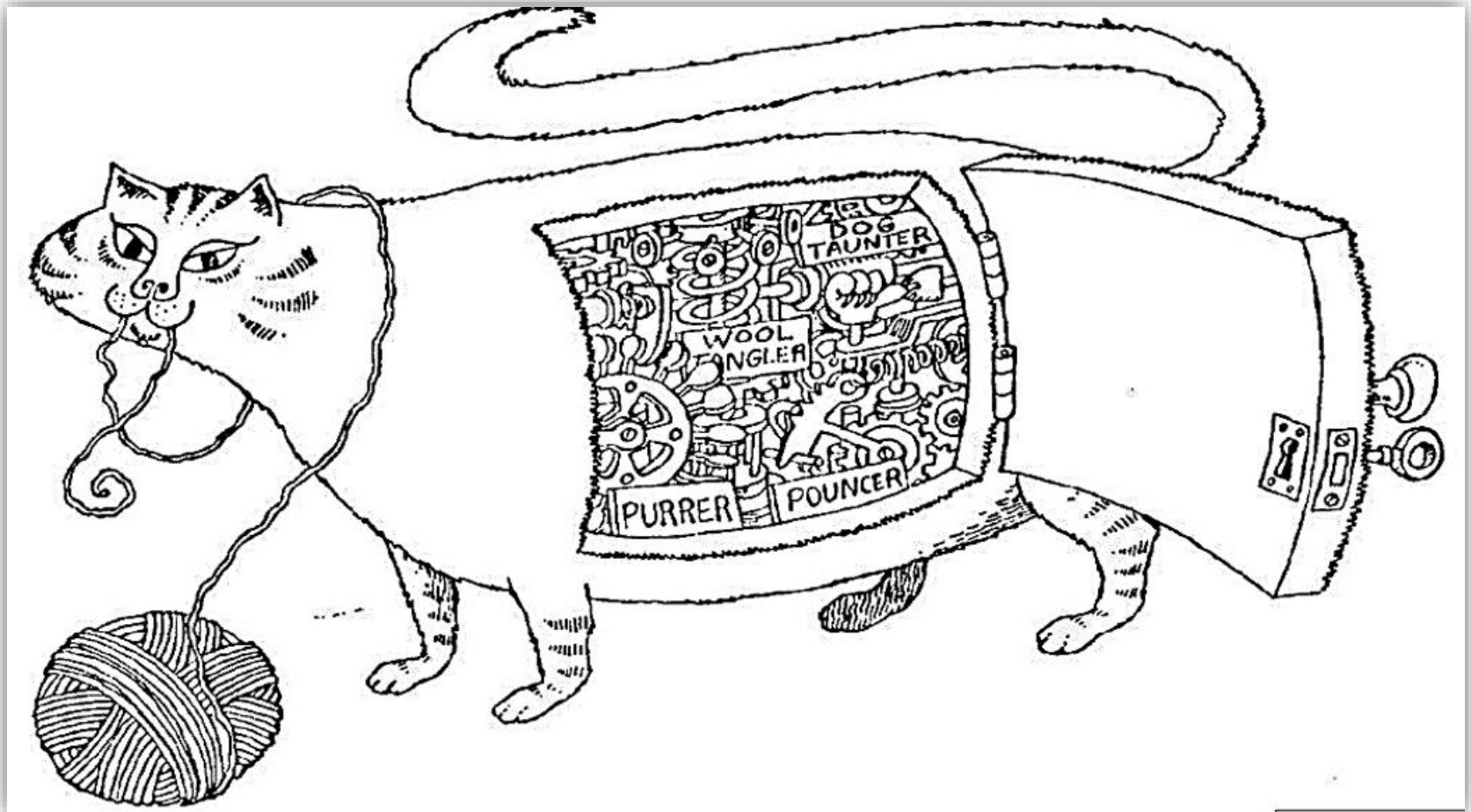
Encapsulamento

- **Resumindo:** “**Não** mostre as cartas de seu baralho” – Utiliza-se o **objeto** sem precisar **conhecer seu conteúdo** (implementação)
- **Objetivos:**
 - **Ocultar** do mundo externo ao objeto os detalhes de implementação e **restringir** o acesso às propriedades e aos métodos;
 - Permitir a criação de programas com **menos erros** e **mais clareza**.
- **Vantagens:**
 - Segurança no **acesso ao objeto**;
 - **Melhor consistência** no estado interno, pois evita **alterações incorretas** nos valores das propriedades.



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Encapsulamento



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Herança

- **Resumindo:** “Filho de peixe, peixe é”.
- Permite definir **novas classes** (subclasses) a partir de uma **classe já existente** (superclasse).
- A subclasse **herda** as **propriedades comuns** da **superclasse** e pode ainda **adicionar novos métodos** ou **reescrever métodos herdados**.
- **Objetivo:** evitar que **classes** que possuam atributos ou métodos **semelhantes** sejam **repetidamente criados**.



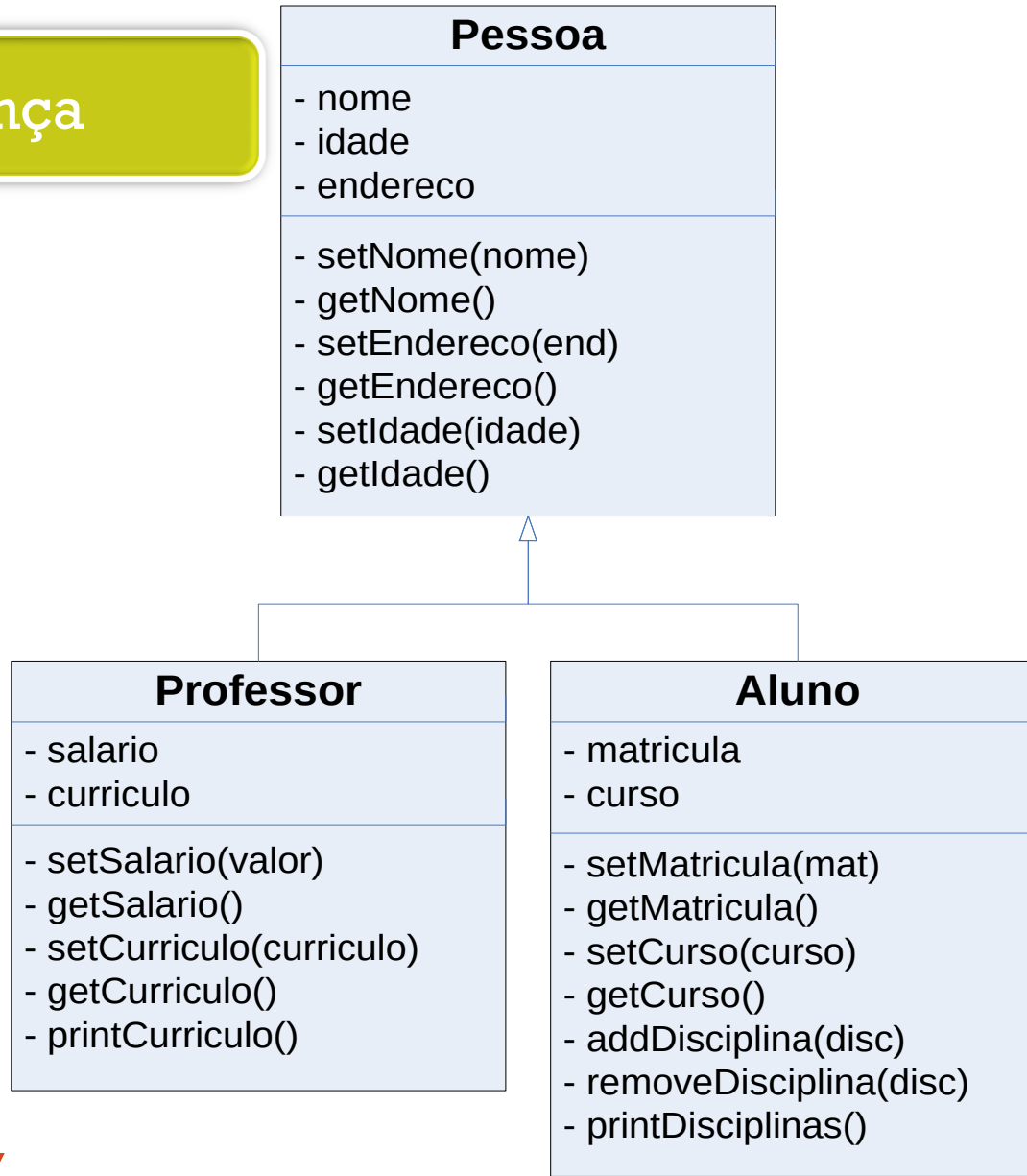
PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Herança

HERANÇA SIMPLES

Especialização

Generalização



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

HERANÇA MULTIPLA

Relógio



Rádio



Rádio-Relógio



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Polimorfismo

- permite a utilização da mesma operação entre classes genéricas e específicas, porém com implementações (métodos) diferentes.



Funcionário

Calcular Salário

herda de ←



Gerente

Calcular Salário



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Polimorfismo

Salário Líq. =
Salário Bruto – descontos



Funcionário

Calcular Salário

herda de



Gerente

Calcular Salário



PRINCÍPIOS BÁSICOS DE ORIENTAÇÃO OBJETO

Polimorfismo

Salário Líq. =
(Salário Bruto + **Comissão**)
– descontos



Funcionário

Calcular Salário

herda de



Gerente

Calcular Salário



CONCLUSÃO



CONCLUSÃO

“Orientação a Objetos é o futuro,
e o futuro é aqui e agora”.

Edward Yourdon

“É hora de **uma revolução na forma de
construirmos software**, A revolução é a
programação orientada a objetos”.

Brad J. Cox

“Provavelmente o software após o ano 2000 você não
estará desenvolvendo, você estará herdando, conectando
componentes e distribuindo o processamento”.

REFERÊNCIAS

- FURLAN, José David. Modelagem de Objetos através da UML: análise e desenho orientados a objeto. São Paulo, Makron Books, 1998.
- Leandro, K. Programação Orientada a Objetos - Uma Introdução. Disponível em http://www2.dca.fee.unicamp.br/pesquisa/jornalLCA/centro.php?sesso_id=&login=&estilo=&conteudo_id=52 >. Acesso em 23 de março de 2022.

